

AI應用開發工程師_光紅建聖_面試資料_如何準備以最大化錄取機率

逐一把這 7 點「想透」，並把每一點從「該做什麼」推進到「具體怎麼做、講什麼」。先講一個貫穿全部的原則：這家公司缺的不是「會用工具的人」，是「能對長期專案負責、能定義方向的人」。所以你準備的每一句話，最終都要能回答主管心裡那句「這個人能不能扛三年？」

第 1 點：重講兩個專案——從「功能清單」改成「決策日誌」

為什麼這點最關鍵：短專案看「你做了什麼」，長專案看「你在岔路口怎麼選」。主管聽你講功能，只能判斷你的技術寬度；聽你講取捨，才能判斷你的工程成熟度與「會不會把架構債留給後人」。HR 講的「完整度」，本質就是在問這個。

做法：把每個專案整理成 3-4 個「決策點」，每個決策點套同一個結構：

情境 → 有哪些選項 → 我選了什麼、為什麼 → 代價是什麼 → 結果/後來怎麼修正。

以你的 DocAI 為例，你手上至少有這幾個現成的決策點可以講：

- **為什麼 Milvus + FAISS 雙架構，而不是單選一個？** 這是最好的取捨故事。FAISS 在單機、純記憶體、低延遲、零服務維運成本上贏；Milvus 在跨機器共享、持久化、可擴展上贏。你選雙架構，代表你在「單機快速」與「跨機協作」之間做了**情境分流**，而不是無腦上重武器。這句話一講出來，主管就知道你不是只會抄教學文。
- **切分策略 (chunking) 怎麼定？** 固定長度 vs 語意切分 vs recursive，chunk size 與 overlap 怎麼權衡——切太細語意斷裂、切太粗檢索不準。講你「怎麼調、用什麼判斷調好了」。
- **為什麼選 BGE-M3 做 embedding？** 這點對台灣製造業特別加分：BGE-M3 的中文與多語檢索能力正好對應他們繁中、跨國（台/寧波/捷克/菲）的文件場景。把技術選型connect到他們的真實處境。
- **跨機器知識共享是怎麼設計的？** 這是長期維護的證據——你不是做完 demo 就丟，而是設計了讓多台機器共用知識的架構。

TeacherAssist.AI 你有一個更強的武器：從 **Presenton 遷移到 PPTAgent 的重構**。這題直接是「長專案決策延續性」的活教材，準備講清楚「原本哪個決策後來變成包袱、所以重構時怎麼設計才不重蹈覆轍」。

關於數字： 我不知道你專案的真實數據，所以不要用我編的——但你面試前一定要把這些「指標欄位」填上你的真實數字（哪怕是估計）：檢索的 recall/命中率改善幾 %、回應延遲從幾秒降到幾秒、文件處理量、維護了多久/迭代幾個版本、token 或成本省了多少。**有數字的取捨 = 可信；沒數字的取捨 = 嘴砲。** 就算只是「P95 延遲從約 8 秒降到 3 秒」這種量級，也遠勝於「我有優化效能」。

第 2 點：補上「成本 / 穩定性 / 評估」這條線（你最該補強的一塊）

為什麼： 這是工程師普遍最弱、企業最在乎的落差。JD 把它列成獨立職責（第 3、4 條），代表這是篩選重點。多數應徵者能講 demo，但講不出「上線後怎麼確保它不退化、不燒錢、不亂答」。你只要能講，就贏一大半人。

好消息是你有一個被低估的王牌： 你做過語音發音評測（GOP / MDD / APA、logit-based 的統計評測方法）。這本身就是一套「為不確定的模型輸出建立量化評估標準」的經驗——這跟 Agent 評估是同一種能力，只是換了領域。面試時務必把它包裝成「我長期在做的就是替模型輸出建立可量化的評估體系」，這比「我會用 LangSmith」有說服力得多。

三條線各準備一個具體答案：

- **評估：** 建一組代表性的 eval set（測試案例 + 期望標準）→ 用 LLM-as-judge 或人工標註評「任務成功率、忠實度（答案有沒有依據）」→ 每次改 prompt/換模型都跑回歸測試，確保沒退化。一句話：「我把 Agent 當成要做回歸測試的軟體，而不是每次靠手感。」
 - **成本：** 模型分級（簡單任務用小模型/地端開源，難的才呼叫大模型）、快取重複查詢、精簡 context、非即時用批次。一句話：「我會先量測錢花在哪一步，再針對最貴的環節砍，而不是憑感覺省。」
 - **穩定性：** 對關鍵輸出用 structured output / function calling 強制格式、加驗證層（不合規就重試）、降 temperature、設步數與 fallback 護欄。一句話：「企業要的是穩定可靠，我的工作就是把 LLM 的不確定性關進可控的籠子。」
-

第 3 點：把記憶系統補強（Neo4j 是你唯一明顯的缺口）

為什麼： 你向量庫很熟（Milvus/FAISS/Qdrant/pgvector），但 JD 明確點名 **Neo4j** 知識圖譜。這是 JD 裡少數你 memory 沒顯示實戰經驗的點，主管很可能會戳。

準備三件事就夠應對：

1. **向量 vs 圖譜的本質差異（用一句話定位）：** 向量庫解決「語意相似」的模糊搜尋（我只記得大概意思，幫我找相關的）；知識圖譜解決「明確關係」的多跳推理（A 供貨給 B、B 又跟 C 有合作，這種關聯鏈）。
2. **什麼情境該用哪個：** 製造業很多知識是關係網（客戶—訂單—料件—製程—良率），這種「誰影響誰」的問題向量庫答不好，圖譜才行。所以兩者互補，不是二選一。
3. **GraphRAG 概念：** 查詢時先用圖譜鎖定相關實體與關係，再用向量補語意細節——「先用地圖定位區域，再用放大鏡看細節」。

誠實框架（如果被問「你 Neo4j 實戰經驗？」）： 不要假裝精通。建議這樣答：「我向量檢索與 RAG 架構是實戰主力，Neo4j 我理解它的定位與 GraphRAG 的整合方式，但還沒在 production 大規模操作過——這正是我想在這個職位深化的部分。」**坦白 + 展現你知道為什麼要學它，比硬吹更可信。** 面試前真的去 Neo4j 官網跑一個 30 分鐘的 hello world，你就能講得更踏實。

第 4 點：前 90 天藍圖（你最高槓桿的一張牌）

為什麼： 這直接補他們最大的洞——「定義 AI 長期發展藍圖」。一個剛起步、沒人做過 AI 的公司，最焦慮的就是「進來的人到底打算怎麼開始」。你能講出有節奏的計畫，主管腦中就會自動把你放進「能扛起來」那一格。

直接給你一個可以套用的 30-60-90 結構：

- **0-30 天 | 盤點與信任：** 不急著寫 code。先訪談各部門（業務、廠務、品保、供應鏈），盤出「高頻、痛、但出錯不致命」的候選場景；同時摸清現有系統、資料散在哪、權限怎麼管。產出一份「AI 落地機會地圖」。
- **30-60 天 | 一個能被看見的 PoC：** 從盤點中挑一個低風險高價值場景做出可動的雛形，並**同步建立評估基準**（這個 Agent 怎麼算成功）。重點是「做出一個能被內部看見、敢用的小勝仗」，替後續鋪信任。
- **60-90 天 | 可複製的方法與藍圖：** 把 PoC 的開發流程、評估機制、記憶系統雛形（pgvector 起步、預留 Neo4j）整理成「可複製的模板」，並提出後續 6-12 個月的橫向擴展藍圖。

講這段時的關鍵心法： 強調「**先建信任、再擴規模**」，而不是「我一進來就要上大系統」。傳產對 AI 又期待又怕受傷，你展現的節奏感本身就是專業度。

第 5 點：製造業 domain 功課（讓技術接地氣）

為什麼：多數 AI 工程師講的都是通用場景，講不出「你們公司能用 AI 幹嘛」。你只要準備 2-3 個貼著他們業務的場景，就會顯得「這個人已經在替我們想了」。

結合他們的實況（光通訊元件、台/寧波/捷克/菲多廠、AI 資料中心客戶、供應鏈），準備這幾個：

- **跨廠品質異常彙整 Agent：**四個廠的品質報告格式、語言可能都不同，人工彙整很花時間。Agent 自動把各廠異常彙整成統一摘要 + 趨勢，主管一眼看完。痛點明確、出錯不致命，是好的第一個 PoC 候選。
- **業務報價 / 歷史案例輔助：**業務報價時，用自然語言查「這個客戶、這個料號的歷史報價與品質紀錄、目前料況」——把散在多系統的資訊一次撈齊。
- **供應鏈料況問答助理：**他們點名 AI 用的 6912 芯光纖原料依賴美日大廠、有缺料雜音。一個能即時回答「某料件目前庫存、在途、替代料源」的查詢 Agent，對採購極有價值。
- **內部知識/SOP 問答（這個最容易落地，也最對應 RAG 你的強項）：**50 年老廠的製程知識、SOP、ISO 文件散落各處，新人查找困難。做一個內部知識 RAG 助理是最低風險的起手式。

講的時候要帶上「為什麼適合 Agent」：這幾個的共同點是「資訊散落多系統、人工彙整耗時、規則清楚」——這正是 Agent 的甜蜜點。能講出這個共通邏輯，比背三個案例更顯功力。

第 6 點：態度定位 —— 拓荒者 + 長跑者（貫穿全場的主旋律）

為什麼：這不是某一題的答案，是你整場面試要持續釋放的訊號。因為他們最深的恐懼是「找來一個人，做了半年炫技 demo 就走，留下一堆沒人維護的爛攤子」。

怎麼一致地傳達：

- 講專案時，永遠帶上「後來」——「後來我持續迭代、後來我怎麼優化、維護了多久」。長跑者會講後續，demo 仔只會講發表那一刻。
- 講方法時，永遠帶上「可複製/可維護」——你在乎的是建立流程與標準，不是一次性的酷東西。
- **要避免的反訊號：**不要過度炫耀你玩過多少最新框架、跑過多少論文（那會強化「他愛追新、坐不住」的印象）。把這些能力定位成「為了把東西做穩、做實用」的手段，而不是目的。

一句可以收尾的定位話術：「我這幾年最有成就感的，不是做出多炫的 demo，而是把東西做到真的有人用、能持續跑下去。」

弱點：太資深 (overqualified) —— 這是你最大的真實風險，要主動拆解

為什麼這比技術更危險：技術不足頂多被多問幾題；overqualified 會在主管心裡種下三顆地雷，而且他可能不會明講，只在心裡默默把你刷掉。你要在被問之前，自然地把這三顆雷拆掉：

1. 「他會不會待不住？」（穩定性疑慮）—— 最大那顆。用你真實的價值觀拆：你重視長期、踏實、把技術做到實用（這跟你 memory 裡偏好「實作落地、長期維護、不愛空談」的風格一致，講起來不會假）。明確說：「我找的不是一個跳板，是一個能讓我長期把 AI 從 0 養大的地方。」
2. 「薪資喬不攏？」（成本疑慮）—— 不要在主管面試時主動談錢，但心裡要有底，並準備一句軟化的話：「我清楚這是個務實落地的職位，我看重的是這個從 0 建起、能定義方向的機會本身。薪資我們可以後續一起談一個雙方都合理的數字。」展現你不是來開天價的，降低他的防備。
3. 「資深的人會不會難帶、看不起傳產？」（文化疑慮）—— 用「我來是拓荒、不是來指揮」的姿態化解。強調你願意先蹲下來盤點、先做小場景建信任、尊重廠務與業務的專業。資深但謙遜，是最有殺傷力的組合。

把 overqualified 翻轉成優勢的核心句：「對一個從 0 導入 AI 的公司來說，需要的不是只會做單一功能的人，而是能定義藍圖、又有耐心走完長週期的人——這正好是我 17 年經驗能給的價值。」把「太資深」重新定義成「我能扛拓荒 + 我懂長跑」，威脅就變成了賣點。

要不要我接著幫你做兩件事其中之一：(1) 把 DocAI 和 TeacherAssist.AI 整理成可以直接講出口的「決策日誌逐字稿」（你把真實數字填進去就能用）；或 (2) 把上面的「前 90 天藍圖」做成一頁帶去面試的簡報。